August 24, 2026

ADT LIST

Activity (In Room)

(A) Look for 6 people

	(String)	(String)	(String)	(char)	(int)
no.	Name	id	course	gender	age
1	Amara		BSIT	M	21
2	Patrishia				
3	Chelsy				
4	Danniah				
5	Kurt				
6	Janie				

(B) Insert 1 person (from your seatmate)

no.	Name
1	Thea
2	Amara
3	Patrishia
4	Chelsy
5	Danniah
6	Kurt
7	Janie

(C) Sort the List

no.	Name
1	Amara
2	Chelsy
3	Danniah
4	Janie
5	Kurt
6	Patrishia
7	Thea

(D) Delete the 4th position

no.	Name
1	Amara
2	Chelsy
3	Danniah
4	Kurt
5	Patrishia
6	Thea

September 2, 2026

BIG O NOTATION

RECAP

```
header (          ){
```

error catching:
```
parameter
isEmpty, isFull
```

```
} else {
```

process:
```
for ( )
while( )
```
(for/while loops)

updating:
```
lastNdx ++;
return A;
```

CONCEPT:

Let's say from USC you wanna go to Ayala Center Block (ACB). Which of these routes with the given estimated time you'd arrive the fastest but also less hassle?

USC ⟶ ACB

Task:

walk == ₱0 → cost
1hr → arrival

jeep == ₱15
== 20 mins

angkas == ~~₱~~ 65
== 10 mins

This is similar to the Big O Notation. Depending on how you code your program it's either: it will take time to run or too fast to run.

CODING PREFERENCE:

For example we have 3 for loops

```
for ( n) {
    for ( n) {
        for ( n) {

        }
    }
}
```

// iterates the list
// checks the elements (?)
// shifting

EXAMPLE

```
for ( 10 ) {
    for ( 5 ) {
        for ( 3 ) {

        }
    }
}
```

n^3

$n * n * n == n^3$

150 = 15 × 10

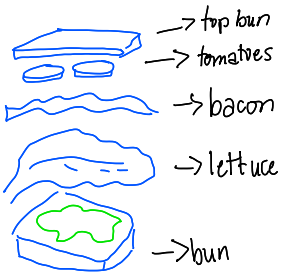
it will run 15 times but iterates for 150 times

$> O(n^2)$

September 2, 2026

STACK

Let's say we want to create a sandwich:



When you make a sandwich you start with the bottom ingredient and slowly go up. Now let's say you wanna add mayonnaise, you remove everything starting from the top and the bottom will be last.

STACK OPERATIONS

```
typedef struct {
    String name;
    String specie;
    String color;
    int breed;
    int age;
} Bird;
```

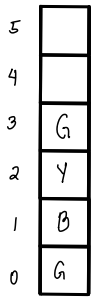
```
typedef struct {
    Bird breadBirds[10];
    int top;
} Stack;
```

void push → insert
void pop → delete
Bird peek → retrieving the specific data

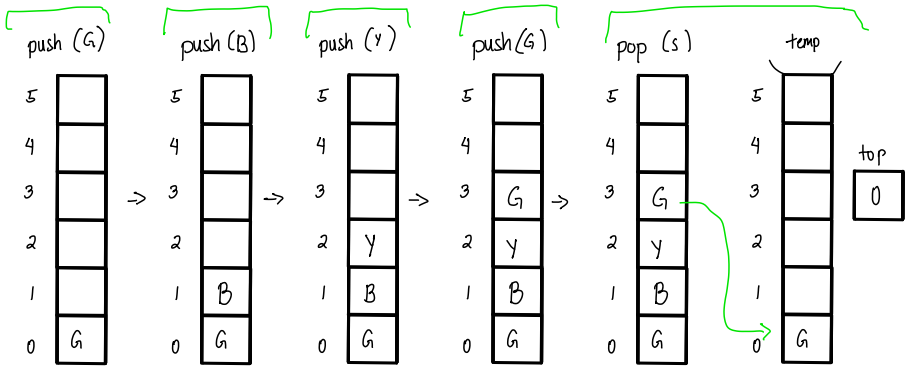
top

Memory of array

G B Y G



pop (s, x)
peek (s) == G
pop (s)
push (temp s, x)



OPERATIONS CONCEPT

Array

```
push (s, x) {
    if( !isFull) {
        s.data[s.top++] = x;
    } else {
        printf("Stack is full");
    }
    return s;
}
```

Ptr

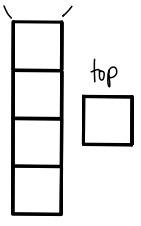
```
push (s, x) {
    if( !isFull) {
        s-> data[s->top++] = x;
    } else {
        printf("Stack is full");
    }
    return s;
}
```

```
pop(s) {
    if (!isFull) {
        s.top--;
    } else {
        printf("Stack is empty");
    }
```

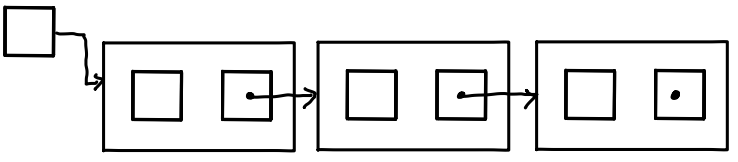
```
pop(s) {
    if (!isFull) {
        s->top--;
    } else {
        printf("Stack is empty");
    }
```

```
typedef struct {
    Birds breadBirds[10];
    int top;
} Stack;
```

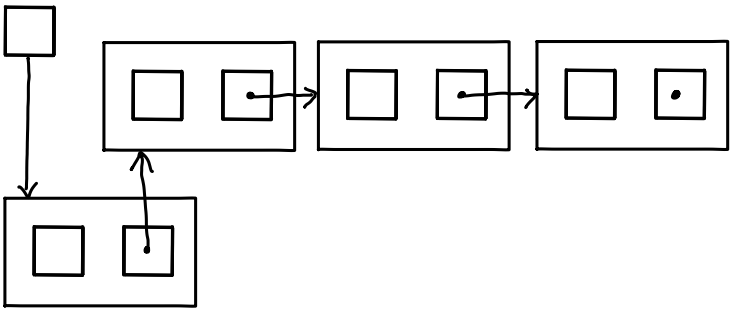
ARRAY



LINKED LIST



OR when inserting a new node:



LIST	Last	First
Array	O(1)	O(n)
Linked List	O(n)	O(1)

ASSIGNMENTS / ACTIVITIES

1.) Linked List activity based on angels

2.) Quiz Gemini

3.) Coding Assignment (Stack)

↳ 1.) push (void), pop, peek (Birds)

only use stack operations → 2.) pushUnique

only use stack operations → 3.) countStack — returns int

Sept. 7, 2026

CIS 2101

STACK PT. 2

(Answering of the assignment)

```
typedef struct {
    String name;
    String specie;
    String color;
    int breed;
    int age;
}
```

Stack:
→ use operations

dili ka access index sa stack unless stated otherwise.

```
typedef struct {
    Bird breedBirds[10];
    int top;
} Stack;
```

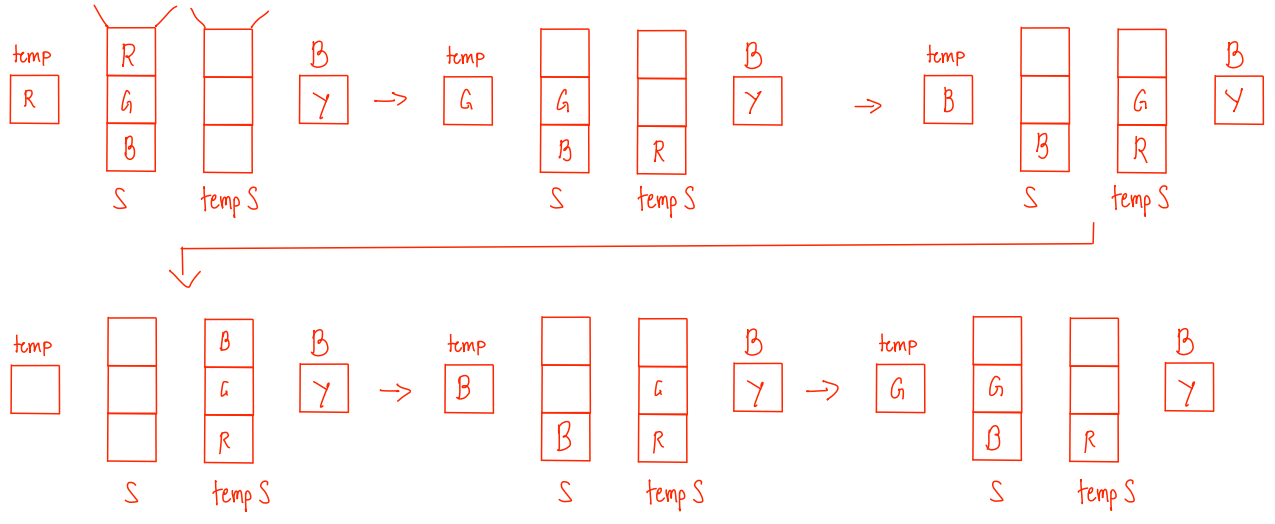
```
void pushUnique(Stack *S, Bird B) {
    if ( S->top > 10) {          // isfull
        printf("Full");
    } else {
        while ( S->top ≥ 0) {    // isEmpty
    Stack? ⇒ temp = peek(S);
what data type? if ( temp -> color != B.color ) {
                push (temp S, temp);   // temp → what's this
                pop (S);
            } else {
                break;
            }
```

note: no need to return since its void

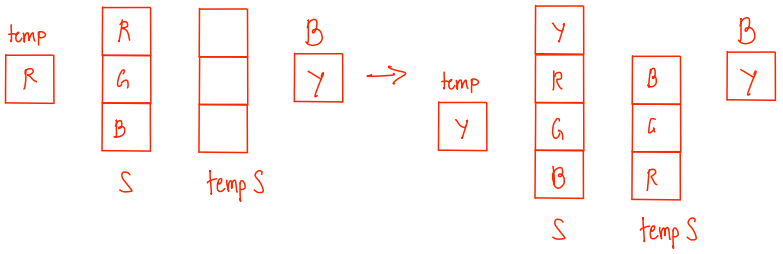
```
void pushUnique(Stack *S, Bird B) {
    if ( S->top > 10) {          // isFull
        printf("Full");
    } else {
        while ( S->top >= 0 &&   // isEmpty
                strcmp(temp->color, B.color) != 0) {
            pop(S);
            push(temp S, temp);
            temp = peek(S);
        }                        // iteration for stack

    if (isEmpty(S)) {
        while (!isEmpty(temp S)) {
            push(S, peek(temp S));
            pop(temp S);
        }
        push(S, B);
    } else {
        while (!isEmpty(temp S)) {   // (since its a pointer return everything to its original state.)
            push(S, peek(temp S));
            pop(temp S);
        }
        push(S, B);
        printf("duplicate found");
    }      I will fix this in the future :)
```

DRAWING FOR THIS PART CAN BE FOUND BELOW

COLOR PATTERN: R -> G -> B -> Y



Something ?



Count Stack → only do the operations.

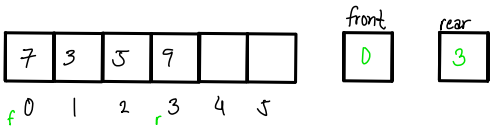
```
int countStack (S) {
    int retVal = 0;
    while (!isEmpty (S)){
        retVal++;
    }
    return retVal;
}
```

← after countStack is done

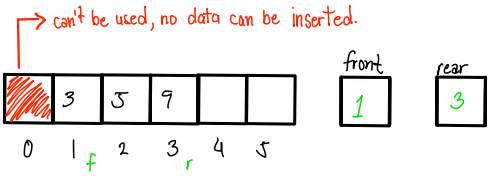
Sept. 7, 2026

QUEUE

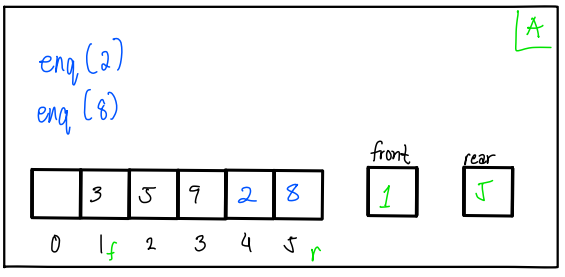
ARRAY IMPLEMENTATION

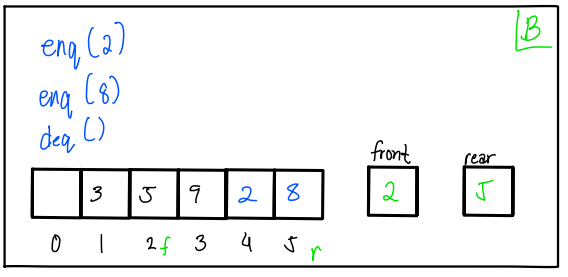


enq(7)
enq(3)
enq(5)
enq(9)
deq( ) →



Now let's say you wanna add the data values: 2 & 8, you have to increment the rear





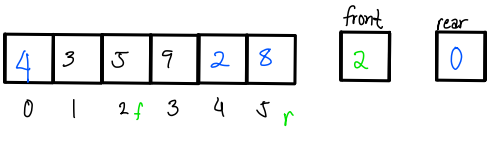
- Now the array is full. We want to insert 4. In order to put the value 4 at index 0, we will use modullos. Since the size of the array is 6, we will divide 4 by 6. Such that:

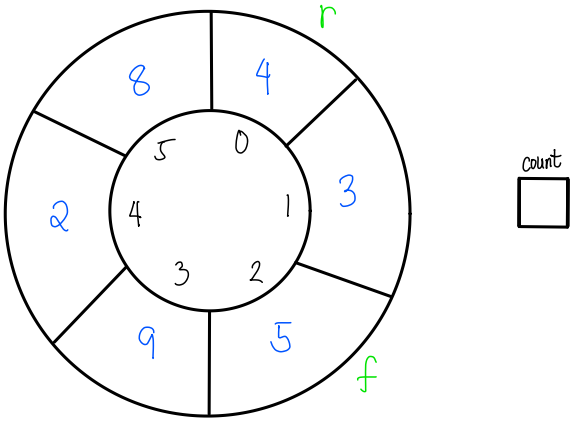
ex1 size: 6 data: 4 ex2 size: 6 data: 3

$$6 \overline{)4}\ 0 \quad R4 \qquad 6 \overline{)3}\ 0 \quad R3$$

↳ Use modullos every time you use the rear

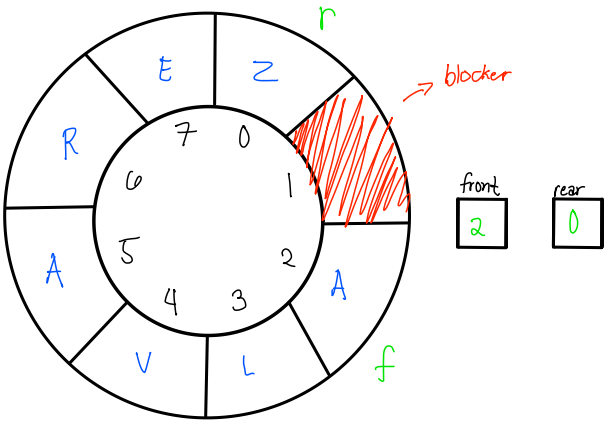
(6)
(rear + 1) % MAX == 0
enq(4);





↳ Another example with size 8.
Now let's insert A, L, V, A, R, E, Z.

enq(A) → enq(R)
enq(L) → enq(E)
enq(V) → enq(Z)
enq(A)



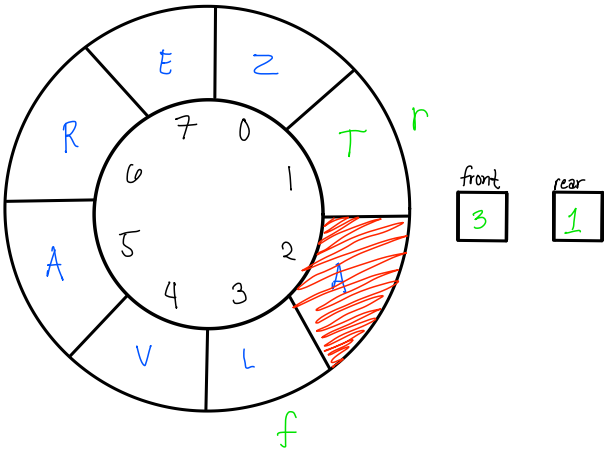
We know that index 0 is the rear, which is the last index.

- Question: How would you know that there is only one element in the array?
  Answer: When your Front is equal to Rear, there is only one element. front == rear ⇒ 1 element

  Question: How would you know that there is an empty list?
  Answer: If Rear is in front of Front, then it is empty. front == r + 1 ⇒ Empty

  Question: How would you know that the array is full?
  Answer: If your Rear is next to the blocker, it's full. front == r + 2 ⇒ Full
  ↳ an indicator that the array is full.

- Now we wanna insert letter T.
  enq(T)



Note:
- In terms of code, you don't have the blocker, it's just a concept.
- Every time you dequeue() the blocker will follow.

ASSIGNMENT

```
typedef struct {
    String name;
    String specie;
    String color;
    int breed;
    int age;
} Bird;
```

```
typedef struct {
    Bird breedBirds[10];
    int start
    int end;
} Queue;
```

Similar to the last Assignment just do it in Queue.

1.) void enqueue; } can access index (?)
    void dequeue;
    Bird front;

2.) void enqueueUnique; (color) } operations only

3.) int countQueue;

Sept. 14, 2026

ASSIGNMENT — ANSWER

```
typedef struct {
    String name;
    String specie;
    String color;
    int breadi;
    int age;
} Bird;
```

Similar to the last Assignment just do it in Queue.

1.) void enqueue; } can access index (?) } these are the operations
    void dequeue;
    Bird front;
2.) void enqueueUnique; (color) } operations only
3.) int countQueue;

```
typedef struct {
    Bird breadBirds[MAX];
    int start    // front
    int end;     // rear
} Queue;
```

ANSWER

```
enqueueUnique (*Q, B) {
    if (Q->start == (Q->end + 2) % MAX) {        // full
        print (" List is full");                 (make block to blue)
    } else {
        temp = front (*Q);
        while (Q->start == ((Q->end + 1) % MAX)) {   // end
            isDuplicate = (strcmp (temp.color, B.color)) 0 : 1;
            dequeue (Q);
            enqueue (&temp Q, temp);
            temp = front (*Q);
        }
        *Q = tempQ;
        if (isDuplicate = 1) {
            printf("Duplicate found");
        } else {
            enqueue (Q, B);
        }
```

process

(rewrite nani)

```
void enqueueUnique (Queue *Q, Bird B) {
    if (Q->start == (Q->end + 2) % MAX) {
        printf("List is Full");
    } else {
        temp = front (*Q);
```

O(n+n) ← has 2 loops

O(n) — only has 1 loop.

Note:
- overall stack needs to be pop & push
- Queue you can just put it in a temp and copy it to the original.

QUEUE:

*Q	temp
A B C D	A B C D

CURSOR - BASED

BOOKSHELF

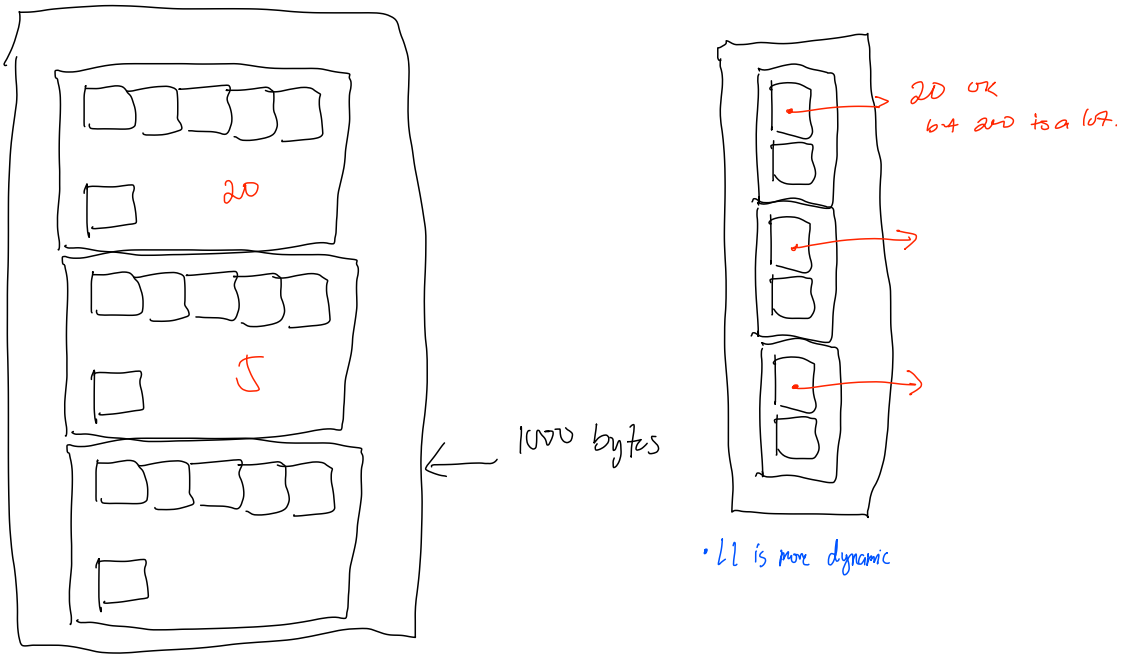
STRUCTURES

Reference the following structures, answer the following prompts with two variations: firstly, access the data without a pointer and secondly access with a pointer.

```
#define MAX 30;
#define char String[50];

typedef struct{
  String bookName;
  String authorName;
  String publishingHouse;
  int publishingYear;
  int totalPageNum;
}Book;
```

```
typedef struct{
  Book books[20];
  int bookCount;
}Shelf;

typedef struct{
  Shelf shelves[3];
  int totalBookCount;
}BookShelf;
```

```
BookShelf BS;
BookShelf *BointR;
```



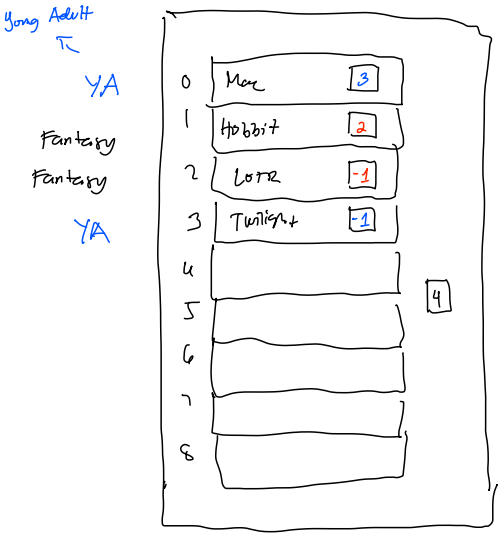
Something that is time efficient & space efficient

↳ causes O(1) instead of O(n)

time efficient ex.

YA 0, 3

FA 1, 2

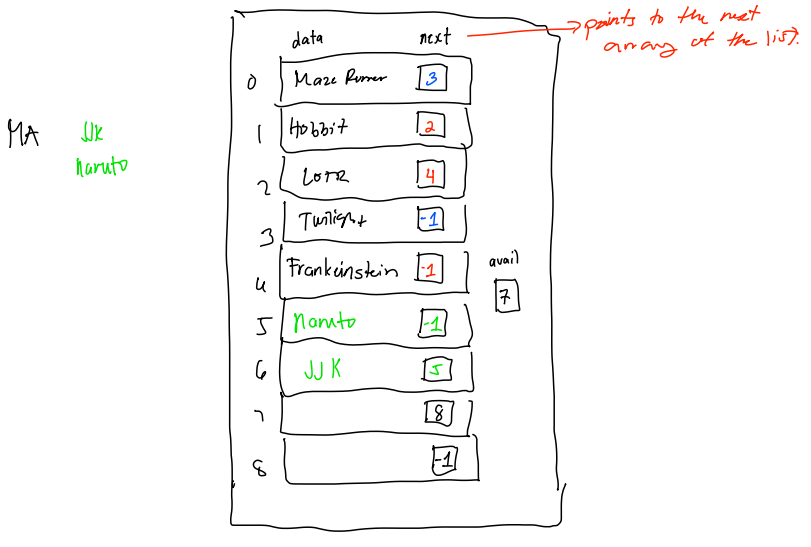


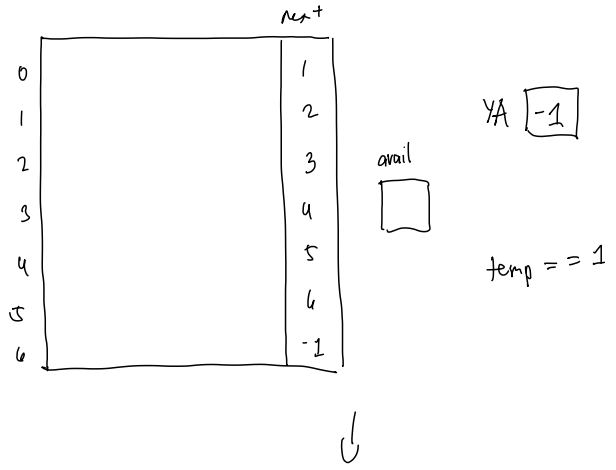
Now I want to unload Frankenstein

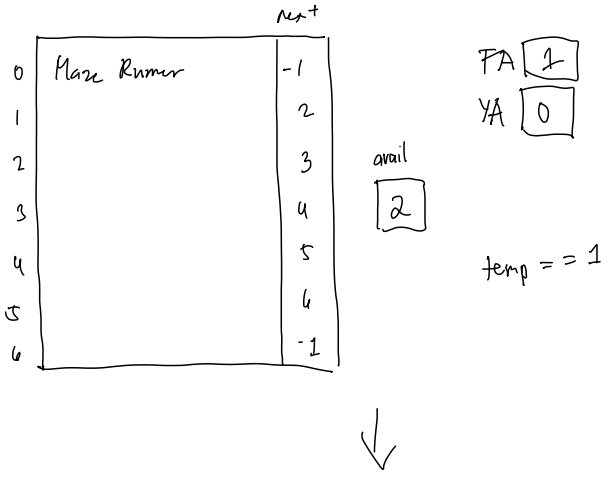
Adding naruto manga

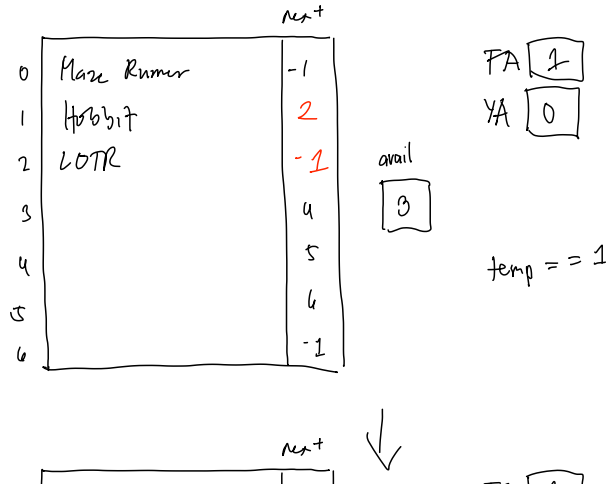
I want JJK to be first

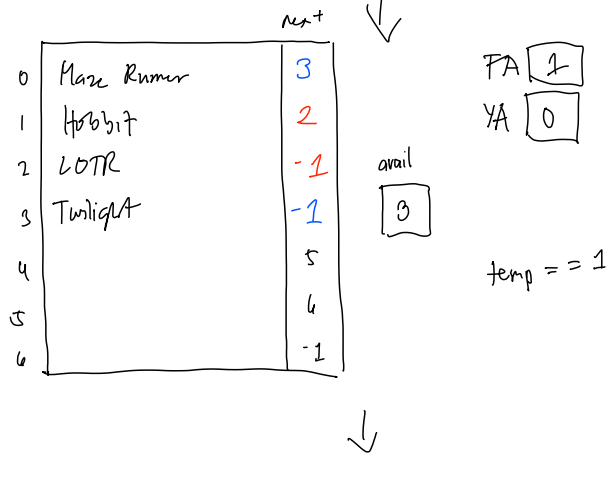
YA 0 PA 1 MA 6

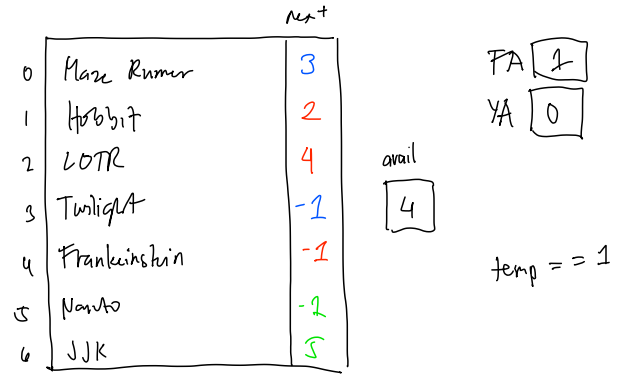



next
0
1
2
3
4
5
6
1
2
3
4
5
6
-1
avail
YA
-1
temp == 1

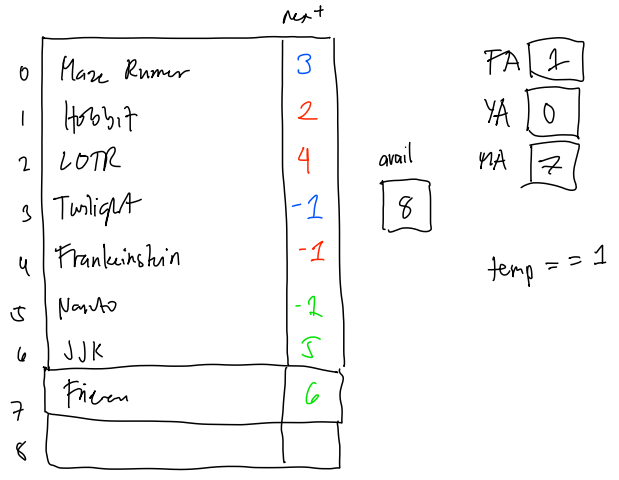

next
0
1
2
3
4
5
6
Maze Runner
-1
2
3
4
5
6
-1
avail
2
FA
1
YA
0
temp == 1

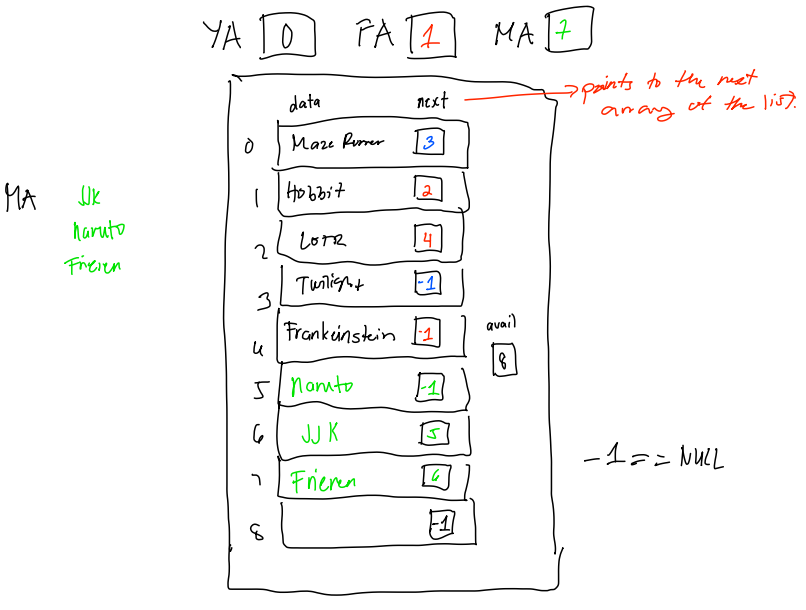

next
0
1
2
3
4
5
6
Maze Runner
Hobbit
LOTR
-1
2
-1
4
5
6
-1
avail
3
FA
1
YA
0
temp == 1

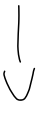

next
0
1
2
3
4
5
6
Maze Runner
Hobbit
LOTR
Twilight
3
2
-1
-1
5
6
-1
avail
3
FA
1
YA
0
temp == 1

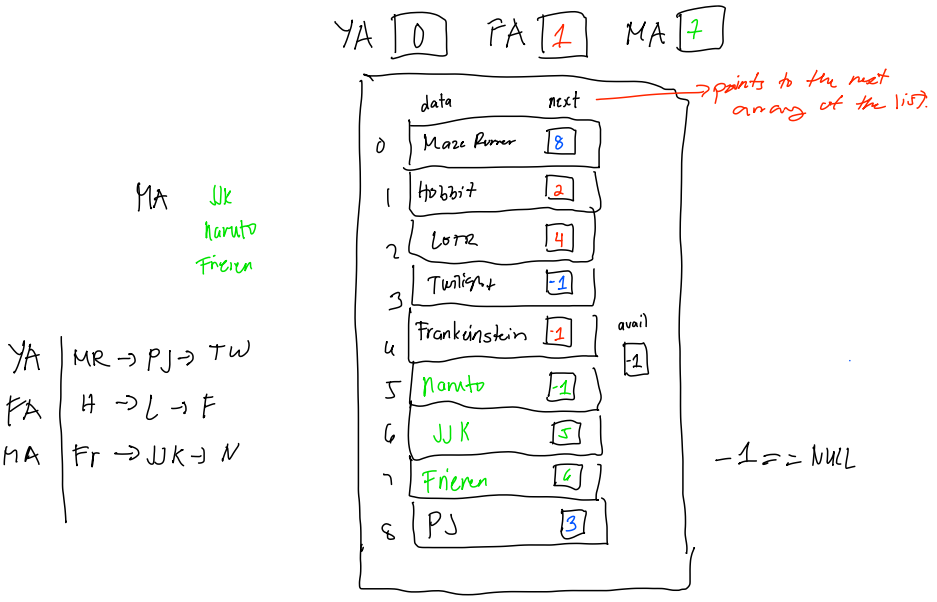


Adding new data

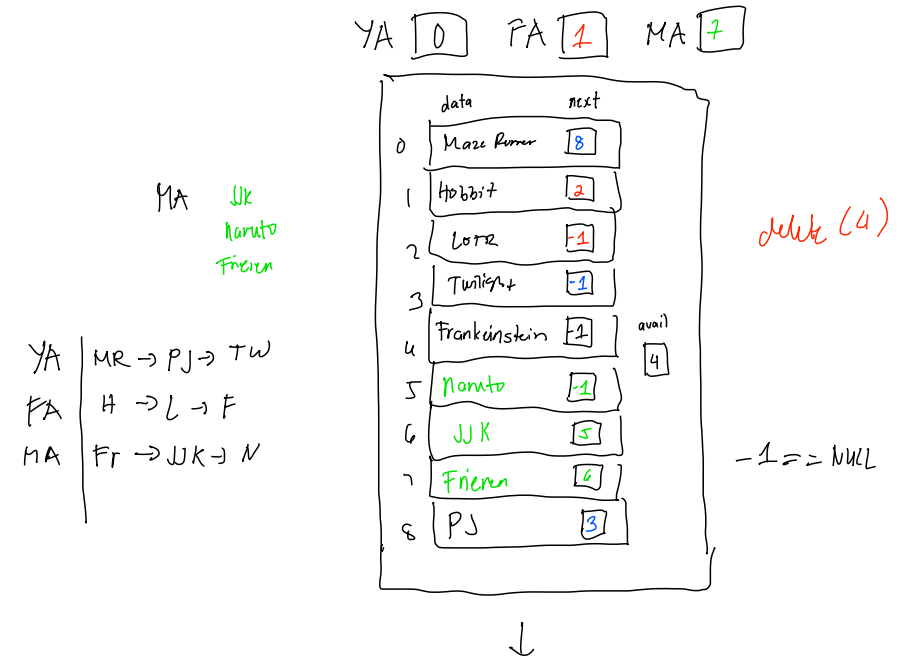




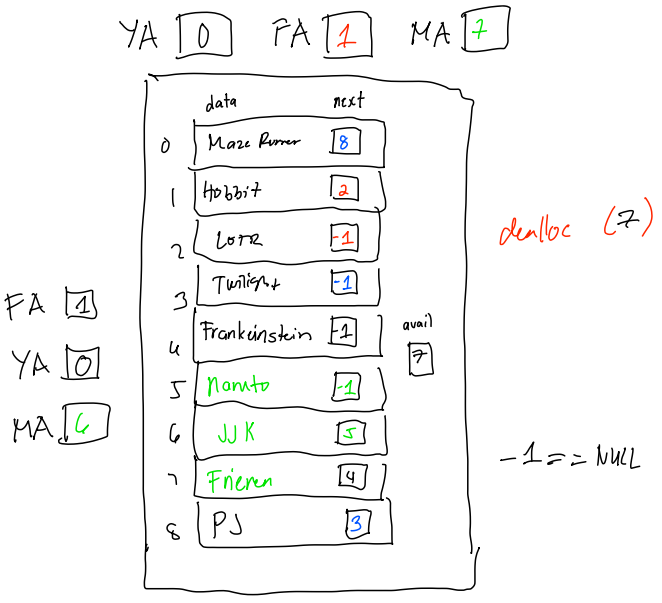




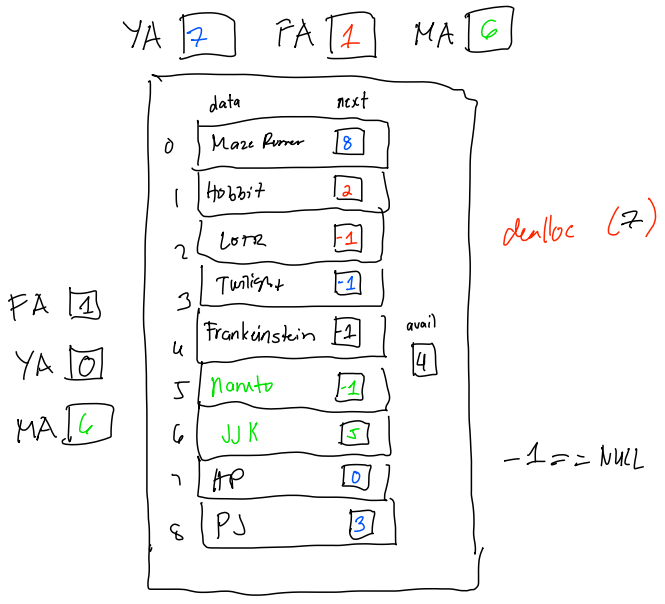
HOW TO DELETE?



DELETE Frieren



Adding Harry Potter



- Use double pointers for CB
- using of next & prev, you can use but not efficient

ASSIGNMENT 3

STRUCTURES

BookShelf

Limit required per puede pa check

Exam : on SATURDAY Sept. 19, 2026

There are 4 parts

1.) ADT List

2.) Stacks

3.) Queues

4.) Circular Based - Basics

↳ more useful in other implementations, than that the more complex.

67

[E1] - Music Playlist - Circular Based

by Edrian Guanzon

in need Sept. 16, work coding

↳ many act related sa exam - Stacks, Queues, List, & CB

09257677990

Call me <3

Sept. 16, 2024

GRADING

CO2 - 12%
Conceptual Topics
- ADT List
- Stacks
- Queues
- Big O Notation

CO1 - 18%
Practical Coding
- ADT List
- Stacks
- Queues
- CB → Borws
  - → 6 test cases
    - all or nothing
      - ij jkg 1.0 or 5.0

1.0 → all test → clt
1.5 → all test → inefficient
2.0 → 90% ~ 99%
2.5 → 60% ~ 90%
3.0 → 60%
3.5 → 10 ~ 60%
4.0 → 1 test case / 0 test case
5.0 → ABSENT

- the total test cases 60% of that you will pass

STACK
- pop, push,

QUEUE
- enqueue, dequeue

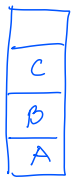
Example of exam

1.) Which of the ff. is the most appropriate disc of a Stack?
A: Data that is stored last is retrieved first
↳ retrieved last (?)
for Queue. Stored last retrieved last

2.) For a pair of a q & a Stack, 2 operations "pop-eq" and "deq-push" are defined as follows:
pop-eq: pops an item fm the stack and puts it into the queue
deq-push: takes an item from the q and pushes it into the stack.

When fm items A, B, C, and D are stored in the q, what is the minimum number of deq-push operations required in order to reverse the order of the four items by using only these two operations.

S deq
Pop



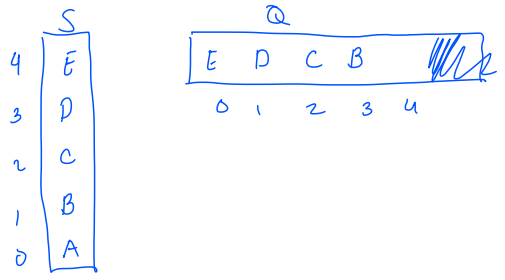
STACK

Q QUEUE

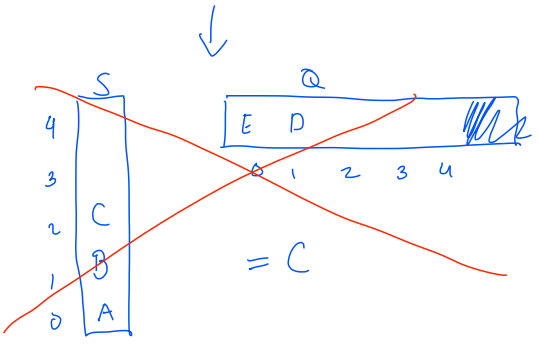
A B C D C B A

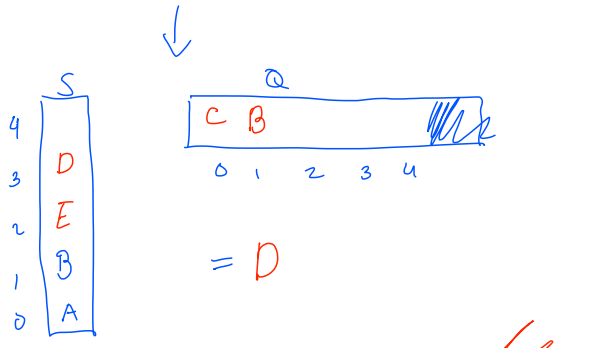
A: 3 ✓

3.) the five items: A, B, C, D, and E are pushed in a stack



A: D





4.) Which of these statements about array list is false

A: the number of elements in the array list has no limit.

- array will always have num. of bytes
- array index is sequential

5.) What is the time complexity of inserting N elements into an initially empty stored sorted linkedlist?

- may linked list pg
- O(N log N) → max in sq trees

A: O(N)

6.) Given the code snippet identify the error

```
//Stack definition
typedef struct
    int items[MAX];
    int top;
} Stack;
// function prototype
void initializeStack(Stack *S);
//code snippet
Stack warehouse[NUM_STACKS];
for(int i=0; i<NUM_STACKS; i++) {
    initializeStack(warehouse[i]);
}
```

A: B. initializeStack(warehouse[i]);

(&warehouse[i]);

7.) Which of the ft. operations can be performed in O(1) time when ADT list is implemented in using LL or any implementation

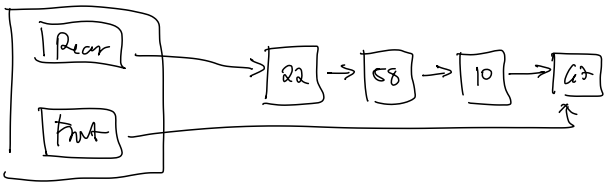
A.) O(n)
B.) O(1)
C.) O(1)
D.) O(n)
Ans: B & C

8.) In the array and LL implementation of list, which of the ft is true?
A: C.) the time complexity of deleting the element

9.) Execution time of an operation is said to be constant if the time it takes to compute...

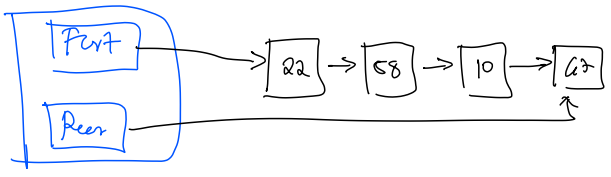
10.)

Queue :



A: time complexity of Enqueue is O(1) and Dequeue is O(1)

if Reverse

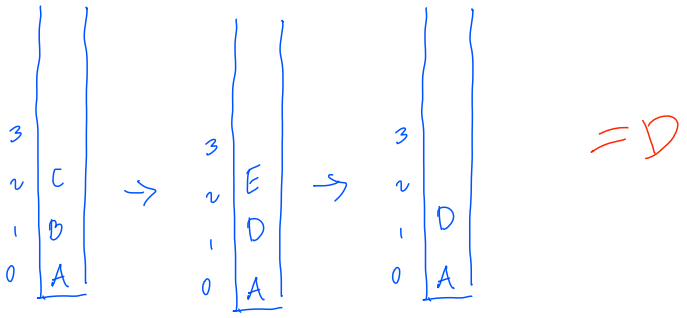


A: time complexity of Enqueue is O(1) and Dequeue is O(1)

Study: Array Implementation

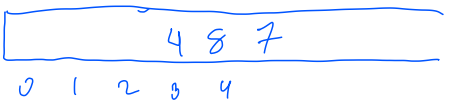
Codelhun Array poster

S



=D

Q



r

f

S